

UNIVERSITI TEKNOLOGI PETRONAS

BACHELOR IN COMPUTER SCIENCE

OBJECT ORIENTED PROGRAMMING (TEB 1043)

GROUP 3

DOCUMENTATION REPORT OF PROJECT

LECTURER'S NAME :

DR. AHMAD FADLI SAAD

PREPARED BY :

N0.	MEMBER'S GROUP	STUDENT ID
1.	MUHAMMAD AQIL BIN MUHAMMAD AMRAN	24005828
2.	NIK MUHAMMAD AQIL BIN NIK SALIMI	25014164
3.	HADIF IRFAN UDA HARRIS	25014026
4.	ZULFA ALEYA BINTI ZUL EMRAN	25014269
5.	AZWA NATASHA BINTI AHMAD SARJI	25014192
6.	NUR ALYAA NABILAH BINTI MAZLAN	25014139
7.	NURUL IRDINA BATRISYIA	25013549

DUE SUBMISSION : 29TH MARCH 2026

Table of Contents

1. Introduction **3**

1.1 Background of the System 3

1.2 Motivation 3

1.3 Project Objectives 3

2. Problem Description **4**

2.1 The Problem being Solved 4

2.2 The System Scope 4

2.3 The Real-World Context 4

3. System Overview **5**

4. UML Class Diagram **6-7**

4.1 Assumption of the UML Diagram 7

5. Use Case Diagram **8**

6. Object Oriented Design **9-12**

6.1 Inheritance 9-10

6.2 Encapsulation 10

6.3 Abstraction 11

6.4 Polymorphism 12

7. System Design Explanation **13**

8. User Interaction / Menu Design **14-15**

9. Game Interface **15-16**

10. Responsibility Breakdown **17**

11. Conclusion **18**

11.1 What the Project Achieved 18

11.2 Key OOP Concepts Implemented 18

11.3 Possible Improvements 18

1. INTRODUCTION

This game application is more to story based adventure game like an AI Dungeon. In Addition, Adventure and Role Playing Games (RPG) are trendy as they allow players to enjoy story while keep moving through different levels fighting enemies and making choice that affect how the game ends. However, many modern RPG use advanced graphics and powerful game engines like Unity and Godot but without they realise actually the basic idea behind all of these games is still built on the simple programming concepts like controlling game flow, managing data and using object-oriented design. So that's why this project is more focused on developing the story text-based adventure RPG game just by using C#, focusing on gameplay logic instead of graphics. By using a console-based approach, the application will highlight the importance of the program structure, class design and interaction between objects in a game system. The system serves as a practical example of how an engaging game can be created by using the fundamental programming techniques without the need for external game engines such as Unity.

Basically, this project is motivated by the idea to practice illustrate the uses of Object-Oriented Programming (OOP) concepts principles in a practical and interactive system. At the same time, we want this game to focus more on the logic and system design compared to visual complexity. For us, by using a text-based adventure game, it can help us to gain more understanding of how an engaging gameplay experience can be built just by using the basic programming techniques that we already learn without relying on the external tools that already exist, like game engines. Without we realise, by doing this project, it also can provide us an opportunity to have a better understanding on how different components in a system can be interact to each other.

The main objectives of this project is to apply Object-Oriented Programming (OOP) concepts in the create, design and development of a text-based adventure game just by using C#. Basically, this project's goal is to show how our group structured and maintainable the system just by using OOP Principles only without the game engines that already exist, like Unity. In Addition, this project also focus on how our group apply the fundamental OOP concepts like encapsulation, inheritance, polymorphism, and more into practice. Furthermore, this project also aims to design a clearer and well-structured system just by using the UML Diagram as a reference before we implement it into a more complex way. Lastly, this project also aims to create a functional game that includes the player's interaction, multiple stages and a progression system. Without they realise, by doing these projects, students can improve their knowledge of OOP Concepts in an actual environment and at the same time they can gain practical experience in designing, implementing and testing a whole system.

2. PROBLEM DESCRIPTION

A Console-based game also adds a great deal of complexity to its development because it requires the handling of several interacting systems such as characters, combat systems, levels, player advancement among others. Although this system interface was already simple if we compared it to other graphical games, we also need to be carefully designed it, especially regarding the internal logic and the system structure of this game so that we can make sure that the gameplay is going smoothly. Without a systematic design, then the system tends to be hard to maintain, extend and debug especially as its features and interactions increase.

One of the major features of a flexible character system design is problematic. The game features distinct entities, such as players and enemies, that have to shared properties like health and attack power but differ both in their behaviour and capabilities. Without hesitation, this would have led to duplication of code and poor scalability when proper abstraction is not in place.

In Addition, the system should also be able to support dynamic fighting interactions. Players must decide as they play, whether they want to attack, defend, heal, or use special abilities, the processing of which must be effective, and at the same time handle game states such as health and mana in real time.

The other challenge is the continuity of a consistent system of progress. The game has to allow different levels, increment in difficulty, narrative flow of the story and the appearance of a final boss, all with the smooth transitions and the consistency in the game logic so that the gameplay experience will be feels more fun to play rather than the story flow being disjointed from one stage to another which make the user turn off to experience it.

In this case, Object-Oriented Programming (OOP) principles are implemented to curb such problems. Modularity Design will play the role of design, creating classes, each with distinct names, namely Character, Player, Enemy, Story, InventorySystem, Item, UpgradeSystem and BattleSystem, which helps reduce redundancy and ensures the design is easily maintainable. This solution allows the interaction of disparate components with a unified interaction with the ability to extend as the solution evolves.

3. SYSTEM OVERVIEW

The Prince Adventure Game is a console-based role-playing game developed by using C# programming language and Object-Oriented Programming concept. The system allows player to control the main character which is the prince in defeating the enemies, final boss and collect items through multiple levels to save the queen. It is based on the decision made by the player where they are able to control the game flow and how they want the story to be. Moreover, the system is designed using the principle of inheritance, abstraction, polymorphism and encapsulation to ensure a structured and clear organization of the code. The system is composed of several main classes that represent the core elements of the game. There are a total of 11 classes in this system which are Player, Enemy, Character, IAttackable, BattleSystem, Item, WeaponItem, GeneralItem, InventorySystem, UpgradeSystem and Story. Classes is made to model complex data structures and behaviors in a modular way where each of the classes have their own methods and attributes.

The Main entities in the system is Player, who represents the main character of a prince with attributes like name, health, attack and mana while Enemy represents the opponents that need to be defeated by the prince with attributes like name, health and attack. The Character base class provides shared functionality of the derived (child) classes. In addition, the system includes an abstract class of Item that is extended into WeaponItem and GeneralItem. WeaponItem stores attributes that enhance the player's abilities to combat, specifically like AttackBonus and ManaBonus, IsEquipped is also included as a boolean flag to track whether the weapon is currently in use while GeneralItem stores attributes that is designed for only one time use like HealthBonus and ManaBonus. The supporting components such as BattleSystem, InventorySystem, Story and UpgradeSystem is added to manage the gameplay structure.

The System is completed with several crucial functions, including managing battles, initializing the game level, handling inventory, distributing rewards and progressing the storyline. Each component plays a major role in ensuring a smooth and well-structured gameplay experience. Furthermore, the system enhances player engagement by providing a graphical image using text-based visuals and narrative progression throughout each level to give a clear vision to the player. At each level, the narrator will explain the situation and update the story flow to keep the player informed and avoid confusion of the player during the gameplay.

Overall, is completed with several crucial functions, including managing battles, initializing the game level, handling inventory, distributing rewards and progressing the storyline. Each component plays a major role in ensuring a smooth and well-structured gameplay experience. Furthermore, the system enhances player engagement by providing a graphical image using text-based visuals and narrative progression throughout each level to give a clear vision to the player. At each level, the narrator will explain the situation and update the story flow to keep the player informed and avoid confusion of the player during the gameplay.

4. UML CLASS DIAGRAM

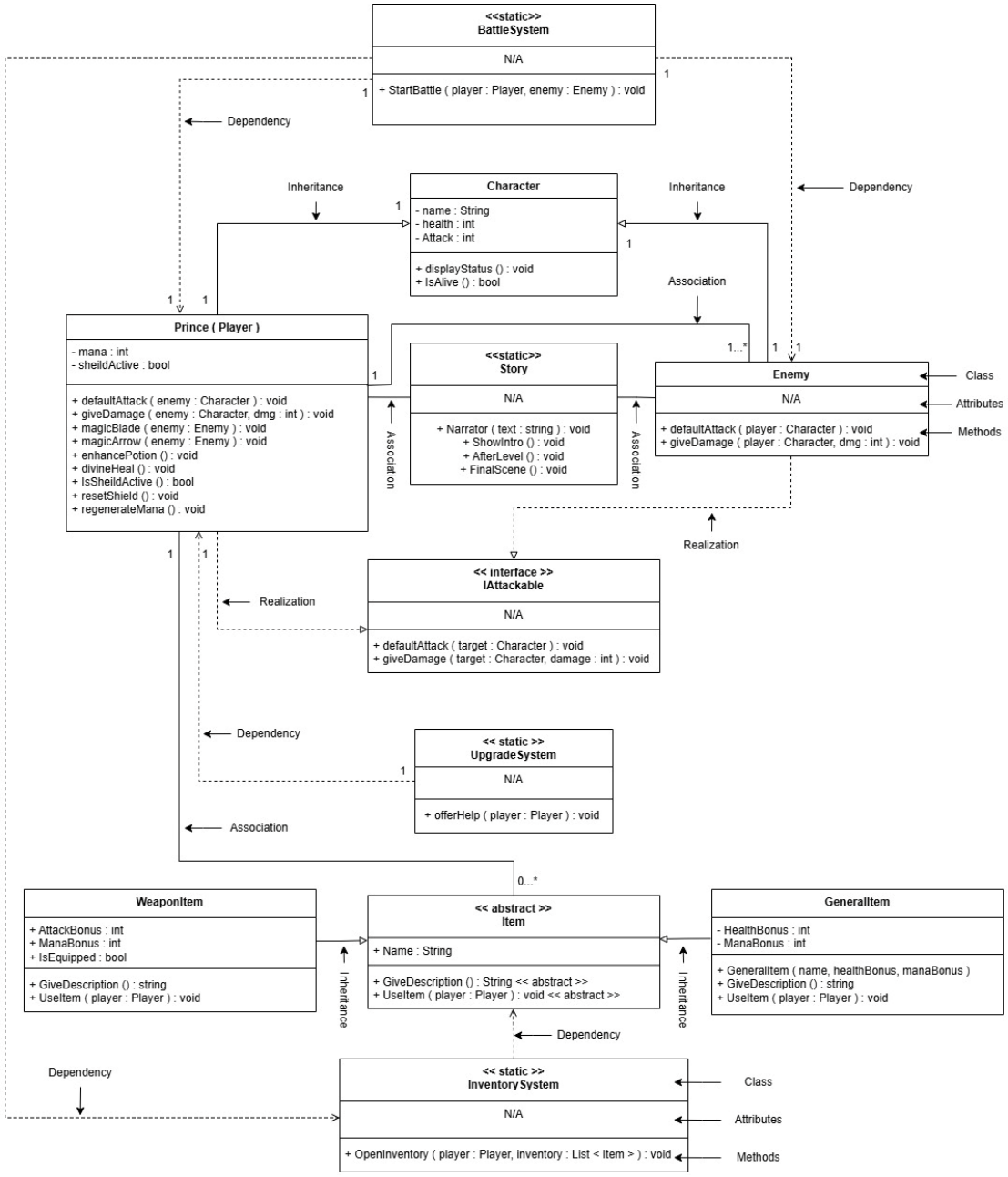


Figure 1 : UML Diagram

ASSUMPTION

Firstly, inheritance is applied to Character Class as to define it as a superclass to the Player Class, while the Player and Enemy Class will be act as the subclass to the Character Class. Both Enemy and Player Class will be inherits all the common attributes and behaviour from Character Class which can make the code of the system reuse it back and reduce redundancy.

Other than that, both Player and Enemy Class will be implement the attack behaviours within the IAttackable interface, as to make sure that there's a consistent structure for the attack related behavious which demonstrate the polymorphism through the realization interface.

In Addition, the Story Class will be associated with the Player Class and interacts with the Enemy Class sequentially as the game progress, as we want to make sure that the narrative flow of the story, including the introduction, progression after each of the stage and the final scenes is maintained throughout the gameplay.

Furthermore, the Battle System Class will be depends to the both Player and Enemy Class, as it required the instances of both these classes to execute the combat operations via the StartBattle () method.

However, this system also apply an abstraction for the Item Class as to define it as abstract class. At the same time, the WeaponItem and GeneralItem Class will be extend the Item Class and provide it the more specific implementations of their abstract methods such as the item usage and the description.

Meanwhile, the player class will be associated with the multiple Item Objects, which showing the player's inventory, and the InventorySystem Class will be depends on all these items to manage and display the inventory during the gameplay.

Lastly, the Upgarde System Class will be depends on the Player Class as it will be interacts with the Player Class as it can enhance the attributes and support the game progression throughout the methods via Offerhelp () method.

5. USE CASE DIAGRAM

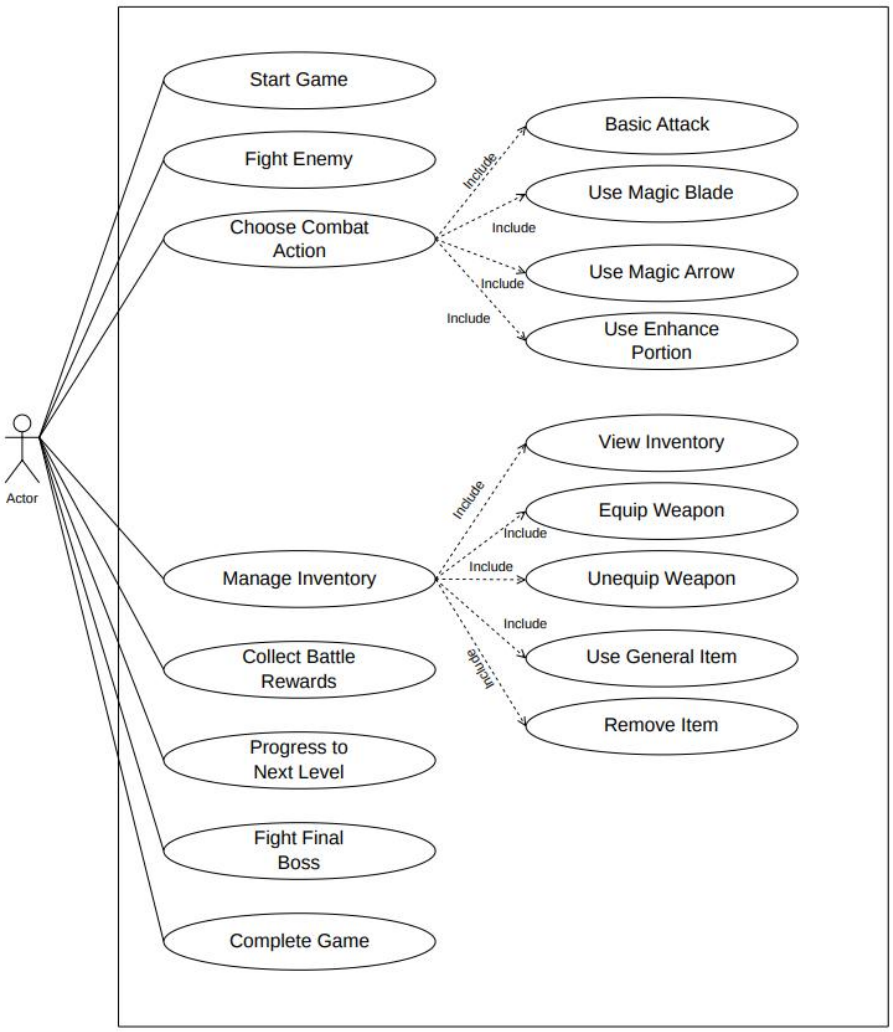


Figure 2 : Use Case Diagram

Basically, this use case diagram is represents the interaction between the player and the game system. The player will be initiates the game, interacts with the story flow, and will be participates in the battle by selecting different combat actions such as a basic attack, use magic blade, use magic arrow and enhance portion. The player also can manage the inventory as they can view inventory, equip any weapon and remove any item that not useful for them. Without we realise, all these interactions help us to define the core functionalities of our game system and help us to illustrate how the user engages with different components in our game system.

6. OBJECT-ORIENTED DESIGN

The OOP Implementations that we used in our system is Inheritance, Encapsulation, Abstraction and Polymorphism.

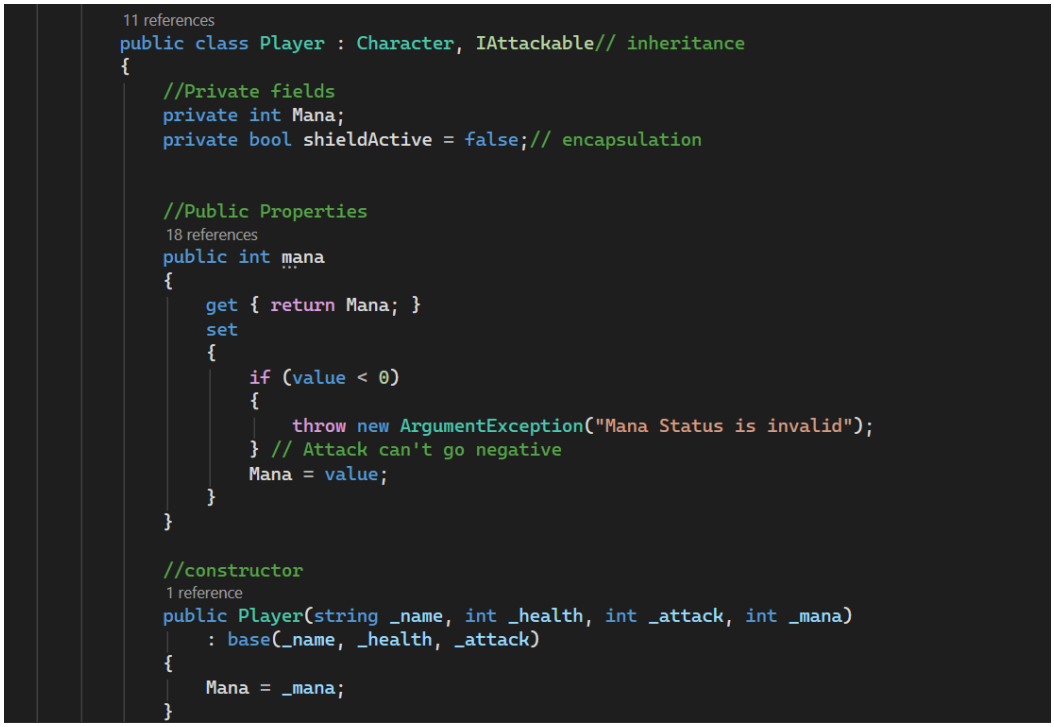
6.1 Inheritance

Inheritance is one of the object-oriented concept where one class will be derives from the other class which want to allow it to reuse the attributes and the methods that already exist.

In our system, one of the part that we demonstrated the inheritance concept is through the Player Class which is this class will be inherits form the Character Class since Character Class will be defines the common attributes such as name, mana, and attack, as well as the methods like IsAlive() and DisplayStatus().

Without we realise, by inheritance from the Character Class, automatically the Player Class will be receive all these properties and methods which can avoid the duplication code happen. At the same time, the player class also will be extend their functionality just by adding the new attributes such as mana and additional methods like MagicBlade() and DivineHeal().

Basically, this concept promotes us to reuse the code instead of rewrite it back many time and allow the different types of class like player and enemy to share the common functionality while maintaining their own behaviour.



```
11 references
public class Player : Character, IAttackable// inheritance
{
    //Private fields
    private int Mana;
    private bool shieldActive = false;// encapsulation


    //Public Properties
    18 references
    public int mana
    {
        get { return Mana; }
        set
        {
            if (value < 0)
            {
                throw new ArgumentException("Mana Status is invalid");
            } // Attack can't go negative
            Mana = value;
        }
    }

    //constructor
    1 reference
    public Player(string _name, int _health, int _attack, int _mana)
        : base(_name, _health, _attack)
    {
        Mana = _mana;
    }
```

Figure 3 Inheritance

Figure 3 shows that implementation of inheritance where the Player Class inherits the attributes and methods from the Character Class.

6.2 Encapsulation

Encapsulation is a way to combine the data known as attributes and the methods into a single unit which known as class while not allowing direct access to some of the object's components. By apply this concept, it can help us to protect the internal state of the object and make sure that it only can be access through public methods.

The Player class is one of the class that used encapsulation in the system. Basically, this class will be defines the private fields like Mana and sheildActive, which cannot be accessed directly from the outside of the class. As the result, we control the access but through the public properties and methods.

For example, the Mana property will be make sure that the value cannot be set as a negative number due to check the validation inside the setter. Without we realise, by doing this, we can make sure the data integrity secured and easier for us to improve the maintainability just by hiding the internal implementation details.

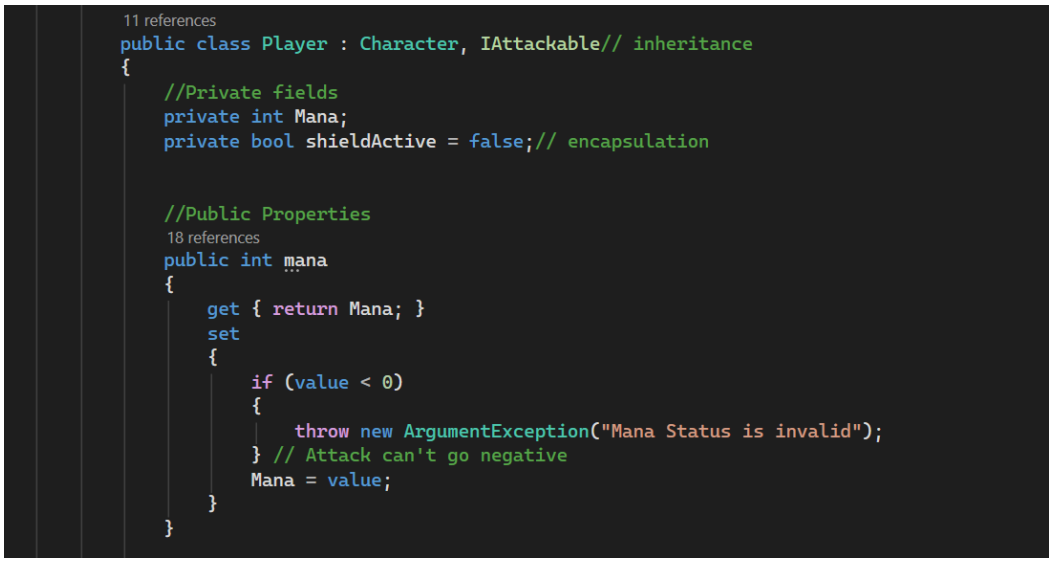


Figure 4 Encapsulation

Figure 4 represents the implementation of encapsulation in the Player Class showing the private attributes and control the access through the public properties.

6.3 Abstraction

Abstraction is the process of hiding the implementation details and exposing the important features of the objects only.

So, in our system, one of the part that we implement the abstraction is by using the abstract class Item. Basically, in this class will be defines the general properties and the methods such as ItemName, GiveDescription() and UseItem(), but it will not provide the full implementation for the methods as abstract class is only define What to do, not How to do it.

As the Result, that's why the subclasses like WeaponItem and GeneralItem will be provide their own specific implementation of all these abstract methods. For instance, the WeaponItem will be increase the attck and mana, while the GeneralItem will be restore the health and mana.

Without we realise, by using abstraction concept in our system, we can make sure all type of items in our game system follow a consistant structure while allowing each of the items act differently.

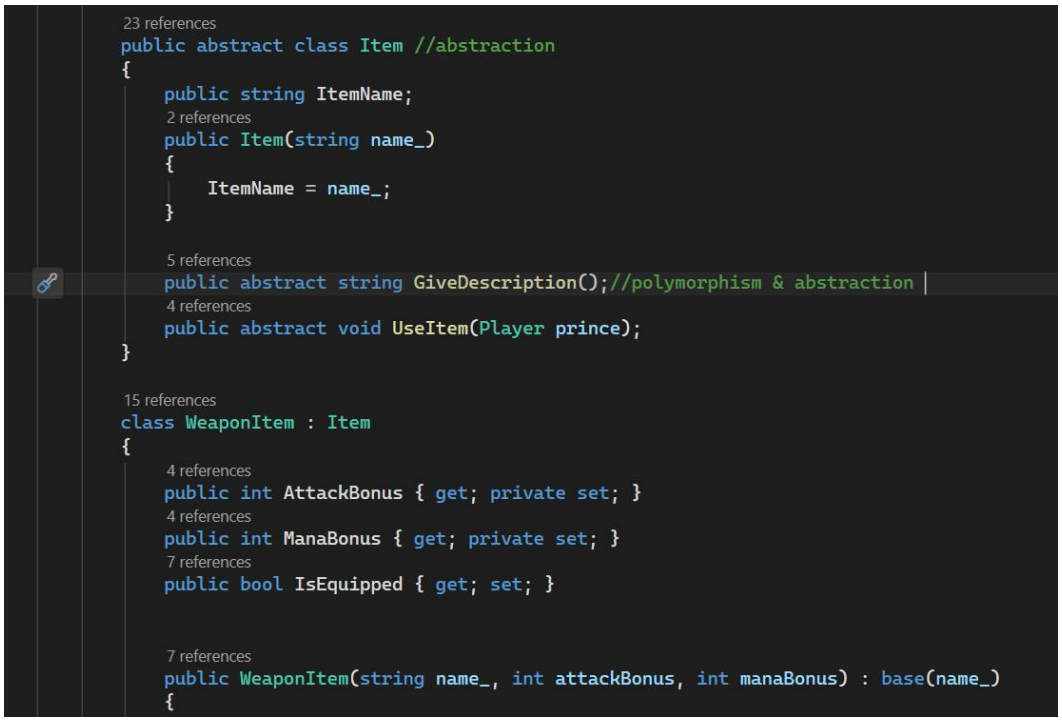


Figure 5 Abstraction

Figure 5 shows implementation of abstraction by using the abstract Item Class where we want to defining the general structure without full implementation.

6.4 Polymorphism

Polymorphism allows us to manage the object as they were instance of their parents class, but at the same time, they also enable the different behaviour just by using the overriding methods.

So, In our System, one of part that we demonstrate the polymorphism concept is through the Item Class and their subclass. Basically, the method UseItem() in this class will be defines as abstract and both class which is WeaponItem and GeneralItem Class will be override it.

Even though both of this class share the same method name, they will be act differently based on the type of the object. Example that we can see is when UseItem() is called on the WeaponItem, it will increase the player's attack and mana in the gameplay, meanwhile when it called on a GeneralItem, it will restore the player's health and mana in the gameplay.

Without we realise, it allow our system to handle the different type of item just by using the commong interface while maintaining the specific functionality for each of the type, which make our system more flexible and scalable.

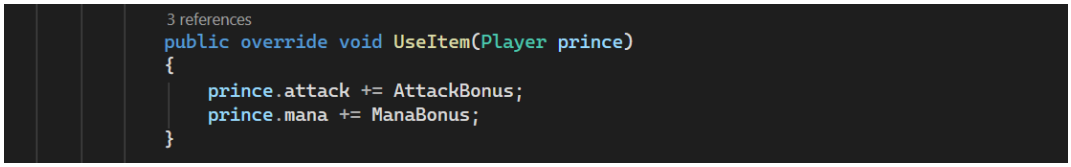


Figure 6 Polymorphism

Figure 6 shows the implementation of polymorphism where the UseItem() method is overridden to provide the different behaviours in the subclasses.

7. SYSTEM DESIGN EXPLANATION

The System design focuses on the relationship between the separate elements to deliver an interactive gameplay. It is a modular design whereby each of the classes has a specific role to play but cooperates with other classes.

The Game flow begins with the BattleSystem that attracts the player and enemy entities and coordinates the game flow, being the key controller of execution of an in-game.

In every interactions, the player is involved in a system of interactions with the adversaries through a set of actions, which include attack, defence, healing and operating special power tasks. These activities directly affect such character attributes as health and mana. These states are constantly checked by the system by the means that are overstated in the Character class, which provides real and correct up-to-dates.

Opponents retaliate by controlling themselves with alternative attack behavior and thus form a turn-based interaction model. As the player continues with the game, the challenge of opponents increases, and more tactical decision-making is required.

UpgradeMaster is presented at identified points to increase player attributes which offers a progression system which points the player to more difficult confrontations, especially the last boss battle.

The very last stage is a face off with a boss, who has better features and higher survival. The system dictates how the game works under such definite conditions: Win condition It means that the boss health will be zero; Lose condition It means that the players health will be zero.

The design ensures that every part functions independent of the other and maintains a smooth interaction. This separation of concerns promotes maintainability, and eases the process of debugging the system besides allowing the developers to extend the system by adding new characters, abilities, or levels without affecting the current functionality.

8. USER INTERACTION

Basically, This game application is a story text-based adventure game where the player will interact with the program through a command-line interface and they will control the character, known as a Prince, who has health, energy, attack power, defence, and healing abilities. At the start of the system, the user will be presented with an introduction of the storyline through the Story Class. At this point, they will provide the context about the prince's mission to rescue the princess. Then, the user can proceeds through the game just by responding to the prompts that will displayed on the screen.

During the gameplay, the user interacts mainly during the battle phase. In each battle, the system presents a set of actions that the user can choose from, such as attacking, defending, healing, or using special abilities. The user selects an action by entering the corresponding option, while the *BattleSystem* will process the input as to determine the outcome of the turn. But, at the same time, the Enemy Class will also respond with their own attack behaviour during the battle. Then, the system will be update the player's and enemy's status, such as health and mana, and display the result to the user.

Other than that, the user also can interact with the inventory during the gameplay trough the InventorySystem Class. By doing this, we can allow the user to see what the available item that they can use to defeat the enemy, read their description, and choose to use them during the battle. At the same time, items also have their own categories which is WeaponItem and GeneralItem. Each of the Category will be provide the different effects during the gameplay and basically, all these item will be used by the player through the defined methods in each of the class.

At each of the battles, the Story Class will be continues the narrative by showing the progression of the scenes, which can be used it as the guidance to the user so that they can proceed to the next stage of the game. But, at certain points, the UpgradeSystem Class wil be interacts with the user for offering the upgrade that can be enhance the user's abilities, which can allowing the user to make the decisions that might be affect their future battles at the next stage.

The game will be continue in a linear progression where the user will be facing multiple enemies since this game consists of four stages, where in each of the level presents stronger enemies over time and they will experience the story progression through the Story Class. Furthermore, since both enemies and final boss have their own abilities such as healing and defending too, that's why the player must think wisely and have an effective strategy to make sure they can be defeated during the battle in each of the level. To prevent the player from overusing the powerful attack, a mana or energy system have to be used to restrict the uses of special attacks so that the player

can use it during attack the final boss. The interaction will be ends when the player either defeats the final enemy and complete the mission or all health drop to zero, which resulting in a game over to the user. A final scene will be displayed to conclude the game.

Basically, the user interaction design of this system is more focus on simplicity and clarity, which mean we just want to ensure that the user can easy to understand how to play this game while remaining engaged in the storyline just by combining the structured menu options with the narrative progression.

9. GAME INTERFACE



Figure 7

Figure 3 shows the example of game interface along side with the narrative storytelling used in the game. The system uses console-based and display the visual using ASCII art to enhance the player's engagement while playing the game.

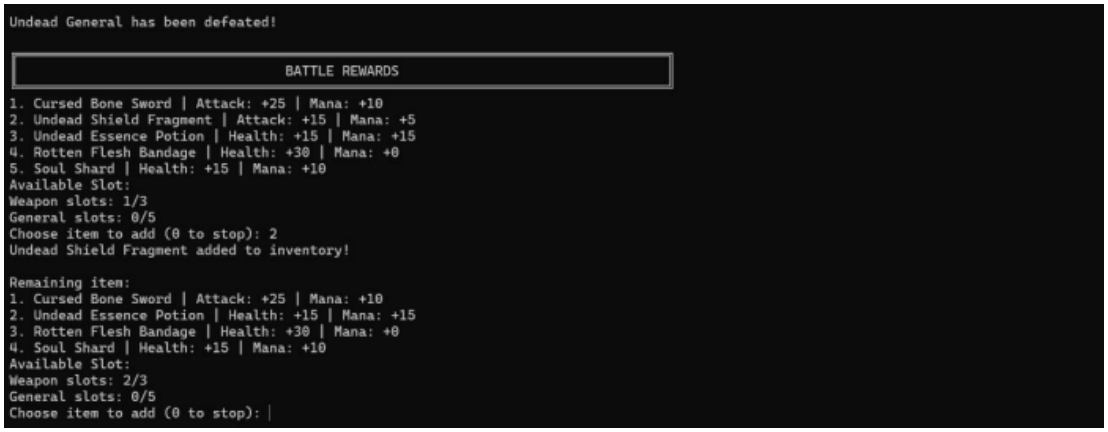


Figure 8

Figure 4 shows the example of the game interface whenever the player (Prince) battle with enemy or the final boss.

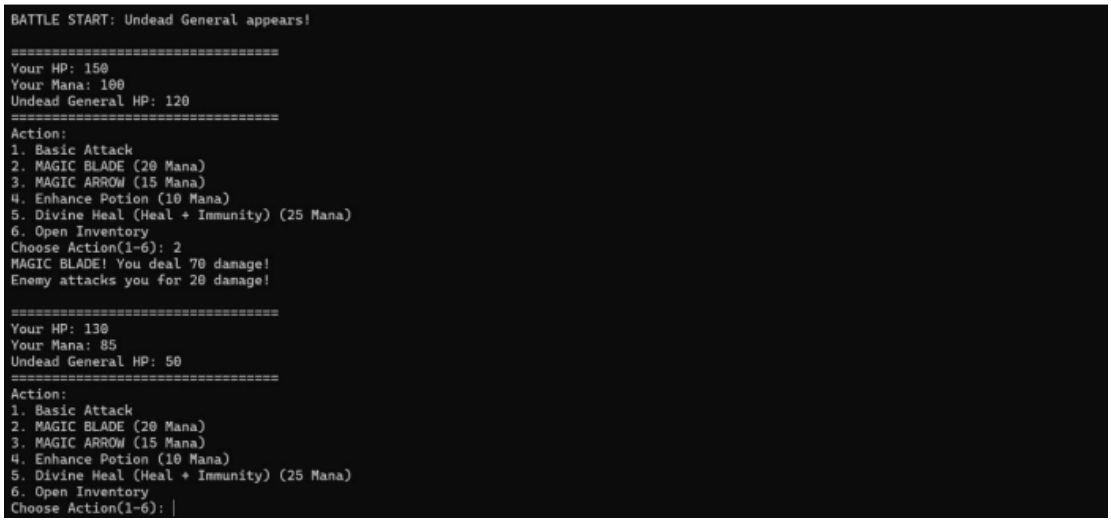


Figure 9

Figure 5 shows the example of the game interface whenever the player defeats the enemy. Battle Rewards will display the recommended weapons or items to be added to the player's inventory.



Figure 10

Figure 6 shows the example of the game interface when the player is defeated.

10. RESPONSIBILITY BREAKDOWN

No.	Task	Student Name	Student ID.
1.	UML Diagram	Nur Alyaa Nabilah binti Mazlan	25014139
2.	Classes Identification	Nik Muhammad Aqil Bin Nik Salimi	25014164
3.	Coding and Testing	Muhammad Aqil Bin Muhammad Amran	24005828
4.	Story Flow	Hadif Irfan Uda Harris	25014026
5.	Documentation	Zulfa Aleya Binti Zul Emran	25014269
6.	Documentation	Azwa Natasha Binti Ahmad Sarji	25014192
7.	Documentation	Nurul Irdina Batrisyia	25013549

Table 1 : Team Responsibility Breakdown Table

All team members contributed effectively to the project. Each student took responsibility for their assigned task. The combined effort and discussion ensured the project was completed successfully and on time.

11. CONCLUSION

In Conclusion, the Prince Adventure Game demonstrated a successful application in terms of core Object-Oriented Programming (OOP) concepts in the functional and interactive console-based game. The use of C# helps the students to develop a game in a way that is easier to understand, making the learning process more fun and accessible. It also encourages students to keep on trying and explore more deeply regarding the OOP concepts, allowing them to continuously improve their skills and create games that are more exciting and higher quality. Throughout the game, the systems shows the implementation of the principles of inheritance, polymorphism and encapsulation, which follows the project objective.

Inheritance is clearly shown through the Character base class, which is extended by both Player and Enemy classes, allowing the reusability of the code, scalability and logical organization where new classes (child) able to inherit the attributes and methods from the existing classes (parent). Moreover, the use of principle of polymorphism can be seen from the use of method overriding, such as DisplayStatus() method, where different classes provide their own output even though using the same method name. Polymorphism allows us to write method in general form, making it easier to extend and maintain the coding. In addition, encapsulation principle is practiced by controlling the access to class properties to manage the game logic safety. The use of private fields and public methods clearly demonstrated this practice.

For future Improvements, the game could include more advanced features such as more mana or power with more impactful effects, richer graphics design and UI elements, making the visual looks more appealing and more engaging, so that players can enjoy it to the fullest. It is also a good opportunity if the game could include animation during the battle between prince and enemies to make it looks more real and enjoyable.

Overall, this project not only achieves the objective in applying the students' understanding of OOP concepts or C# programming but also provides a platform for students to be creative and experience hands-on in designing a structured, functional and interactive application using the OOP principles. It allows students to see how theoretical concepts such as inheritance, polymorphism abstraction and encapsulation are applied in a real world scenario. The project also encourages students to enhance their problem-solving, critical thinking, and time management skills. Furthermore, this project serves as a strong foundation and starting point for more complex, sophisticated and engaging software application development in the future.